Error Propagation in Multi-Step Agentic AI Workflows for Research Automation: A Controlled Simulation Study

Sutariya Gaurav Karsanbhai (MCL2026003)
Department of Computer Science and Engineering
Course paper — AI Tools for Research

***Abstract*—**Multi-step agentic workflows split research automation into stages such as planning, retrieval, evidence extraction and verification. That modularity helps organize the work, but every handoff is also a place where an error can survive and grow. Building on an earlier literature review of error propagation in research-oriented agentic systems, this paper reports a controlled simulation of conditional downstream error propagation. A four-stage workflow is run under two conditions: independent stage errors, and conditional propagation, in which an error at one stage raises the error probability of the next stage by 0.30 (capped at 1.0). Base error probabilities of 0.0, 0.1, 0.2 and 0.3 are tested with 30 random seeds and 100 trials per seed and condition, giving 24,000 workflow records. The raw records were checked for missing values, duplicate rows, schema consistency and label consistency, and all summary statistics were recomputed independently. At the non-zero base rates, propagation raised the mean number of stage errors per workflow by 0.107, 0.207 and 0.331, with paired 95% t-intervals over seed-level means of [0.094, 0.120], [0.189, 0.225] and [0.313, 0.349]. The binary any-error failure rate did not differ between conditions, because that outcome is settled by the first error. The conclusion is therefore a narrow one: under this synthetic error process, propagation increases error accumulation. The experiment does not show anything about real LLM agents, the correctness of research papers, or end-to-end task success. It does provide a reproducible baseline for later work with real agent traces, recovery interventions and terminal-output correctness.

***Index Terms*—**Agentic AI, large language models, error propagation, research automation, fault injection, workflow reliability, simulation.

I. INTRODUCTION

Large language models (LLMs) are now used as parts of workflows that run for much longer than a single conversational turn. An agent may break a goal into subtasks, retrieve outside information, call tools, keep intermediate state and revise its plan as it observes results. In research automation, a typical pipeline covers query planning, scholarly retrieval, source screening, evidence extraction, synthesis, citation and manuscript preparation. ReAct showed how reasoning and acting can be combined, and tool-augmented and retrieval-grounded systems have widened what agents can do [1], [5], [6].

How reliable such a workflow is depends on more than the accuracy of each component. It also depends on the integrity of what is passed between components. An irrelevant source retrieved early may be summarized perfectly well and still yield an irrelevant evidence statement. A later synthesis stage may then treat that statement as established, and a writing stage may attach a valid citation to an unsupported claim. The final document can look fluent and well formatted while its evidence chain is broken. Retrieval augmentation, reflection and multi-agent role specialization can ease particular weaknesses, but none of them guarantees that every intermediate result is correct [7]–[11].

The literature review that precedes this paper sorted research-workflow errors into eight categories: planning, information retrieval, filtering, evidence extraction, reasoning, citing, tool use and memory. It also called for stage-level measurement, causal error lineage, independent verification and reproducible fault-injection experiments. This paper turns that broad review into a small controlled experiment. We do not claim to reproduce a deployed LLM agent. Instead, we isolate one mechanism, conditional downstream error propagation, in a transparent four-stage stochastic simulation.

The study asks how a conditional downstream error rule changes the number of stage errors in a four-stage simulated research workflow, compared with a condition where stage errors stay independent at a fixed base probability. The primary outcome is the mean number of stage errors per workflow. The secondary outcome is the share of workflows with at least one stage error. The difference between the two matters: propagation can add errors after the first one without changing whether a workflow has had any error at all.

II. BACKGROUND AND RELATED WORK

Reasoning and acting. ReAct interleaves reasoning traces with external actions and observations, which gives an agent an explicit loop for interacting with an environment [1]. Chain-of-thought prompting, self-consistency and tree-based reasoning organize intermediate reasoning in different ways [2]–[4]. They motivate multi-stage workflows, but writing out intermediate steps is not the same as verifying them.

Tool use and retrieval grounding. Toolformer and ToolLLM study how language models work with external tools and APIs [5], [6]. Retrieval-augmented generation conditions output on retrieved passages, and Self-RAG adds adaptive retrieval and reflection [7], [8]. These methods supply outside information, yet retrieval quality, parsing and interpretation can still fail. A relevant-looking source, or an API call that ran correctly, does not by itself show that the evidence supports a claim.

Reflection and multi-agent coordination. Reflexion feeds verbal feedback from earlier attempts into later behavior [9]. AutoGen and MetaGPT organize interaction among specialized agents [10], [11], and persistent memory and reusable skills extend agent operation further [12], [13]. Self-critique, however, is not necessarily independent verification, and splitting roles does not guarantee independent error detection when the agents share a model, context or assumptions.

Evaluation. AgentBench, WebArena, Mind2Web, GAIA and SWE-bench evaluate agents on interactive or tool-mediated tasks [14]–[18], and work on citation support and factuality stresses the need to check whether generated claims are backed by evidence [19], [20]. Our experiment is much narrower. It does not run an LLM, browse the web or evaluate citations. It isolates the mathematical effect of one conditional error mechanism so that the consequences can be inspected directly.

III. RESEARCH GAP AND QUESTIONS

The literature review points to a recurring measurement problem. Final task success alone may not show where an error first entered a workflow, how many later stages it touched, or whether the system recovered. A controlled simulation is a manageable first step, since the error mechanism and the outcome definitions can be fixed before any results are seen. It does not close the wider empirical gap, but it can test whether a stated propagation rule changes how stage errors accumulate.

RQ1: At a fixed base per-stage error probability, does conditional downstream propagation raise the mean number of stage errors in a four-stage workflow compared with independent errors? **RQ2:** How does the difference in mean stage-error count change as the base error probability goes from 0.0 to 0.3? **RQ3:** Does the binary any-error failure metric separate the two conditions under the outcome definition used here?

H1: At non-zero base error probabilities, the propagation condition has a higher mean stage-error count than the independent condition. **H2:** The difference in mean stage errors grows across the tested base rates. **H3:** The any-error failure rate is equal between conditions, because both share the same probability of a first error and the propagation rule only affects later opportunities for error.

IV. METHODOLOGY

A. Study Design

The study is a controlled stochastic simulation with two conditions. In the independent condition, every stage has the same fixed error probability p. In the conditional-propagation condition, a stage has error probability p unless the immediately preceding stage had an error, in which case it is $\min(1, p + 0.30)$. Four sequential stages are modeled: planning, retrieval, evidence extraction and verification (Fig. 1). The simulation trains no machine-learning model and uses no external question-answering dataset. HotpotQA was considered in the earlier planning material, but the executable experiment is a synthetic simulator, so no dataset-based or LLM-performance claim is made.

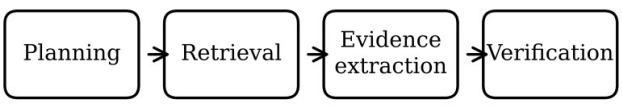


Fig. 1. Four-stage workflow abstraction used in the simulation.

B. Error Model

Let $E_i \in \{0, 1\}$ indicate whether stage i contains an error, and let $p \in \{0, 0.1, 0.2, 0.3\}$ be the base per-stage error probability. In the independent condition, $P(E_i = 1 \mid E_{i-1}) = p$. In the propagation condition, $P(E_i = 1 \mid E_{i-1} = 0) = p$ and $P(E_i = 1 \mid E_{i-1} = 1) = \min(1, p + \delta)$, with $\delta = 0.30$. The first stage uses probability p in both conditions. Each stage outcome is a Bernoulli draw from a pseudorandom generator. The stage-error count is $S = \Sigma_{i=1}^{4} E_i$, and the binary workflow-failure indicator is $F = 1(S > 0)$.

C. Experimental Protocol

The raw file holds 30 seeds (2026–2055), 100 trials per seed, four base error rates and two conditions, which gives $30 \times 100 \times 4 \times 2 = 24{,}000$ records, or 3,000 per rate-condition cell. Both conditions use the same seeds and trial budgets, so the comparison can be paired at the seed level. The primary metric is mean S; the secondary metric is the empirical mean of F. The simulation code and raw CSV are part of the project archive.

D. Validation and Analysis

The validation report covers required files, expected columns, missing values, duplicate rows, allowed error-rate ranges, non-negative integer stage-error counts, and consistency of F with $S > 0$. We also reloaded the raw CSV and re-aggregated it independently for this paper. For the primary comparison, mean stage-error counts were computed within each seed and condition, and the propagation-minus-independent difference was taken for each of the 30 seeds. A two-sided 95% Student t interval was then computed over these 30 paired differences. The binary failure proportions are reported descriptively. No causal claim beyond the specified simulator is made.

E. Reproducibility Environment

The environment log lists Python 3.13.16, NumPy 2.1.3, pandas 2.2.3, PyTorch 2.11.0+cu130 and a Tesla T4 GPU with CUDA available. The simulation itself relies on Python pseudorandom draws and pandas for data handling, so it does not need a GPU. The archive holds notebooks, CSV outputs, validation logs and figures. We did not perform a clean rerun from a fresh environment for this paper.

F. Analytical Expectation

The model has a simple expectation that the empirical means can be checked against. In the independent condition, $E[S] = 4p$. In the propagation condition, let $r_1 = p$ and $r_i = p + \delta r_{i-1}$ for $i = 2, 3, 4$, since the next-stage error probability is $p + \delta$ after an error and p otherwise. Then $E[S] = \Sigma_i r_i$. For $\delta = 0.30$, the theoretical propagation means at $p = 0.1$, 0.2 and 0.3 are 0.5107, 1.0214 and 1.5321. A finite simulation will not match these exactly, but closeness is a useful implementation sanity check.

V. RESULTS

A. Data Integrity

The raw multi-seed file has 24,000 rows and six columns: trial, error_rate, model, stage_errors, workflow_failed and seed. Each of the eight rate-condition cells has 3,000 records. The validation report shows no missing values and no duplicate rows, and a row-level check confirms that workflow_failed is equivalent to stage_errors > 0. All summary values below were recomputed from the raw CSV, not copied from a draft.

B. Stage-Error Accumulation

At $p = 0.0$, both conditions gave zero mean stage errors. At $p = 0.1$, the independent condition averaged 0.3847 stage errors and the

propagation condition 0.4917, a difference of 0.1070. At $p = 0.2$, the means were 0.7887 and 0.9953 (difference 0.2067). At $p = 0.3$, they were 1.1790 and 1.5100 (difference 0.3310). The propagation condition therefore had the higher mean at every non-zero base rate (Table I, Fig. 2).

TABLE I. RECOMPUTED EXPERIMENTAL OUTCOMES

p	Mean ind.	Mean prop.	Diff.	Fail ind.	Fail prop.
0.0	0.0000	0.0000	0.0000	0.0000	0.0000
0.1	0.3847	0.4917	0.1070	0.3360	0.3360
0.2	0.7887	0.9953	0.2067	0.5787	0.5787
0.3	1.1790	1.5100	0.3310	0.7447	0.7447

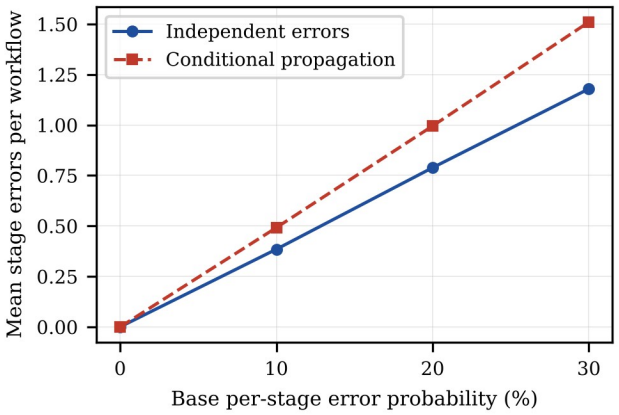


Fig. 2. Mean stage-error count by base per-stage error probability, recomputed from the 24,000-row raw CSV.

C. Paired Uncertainty

Over the 30 seed-level paired differences, the estimated mean increase in stage errors was 0.1070 at $p = 0.1$ (95% t interval 0.0944 to 0.1196), 0.2067 at $p = 0.2$ (0.1888 to 0.2245) and 0.3310 at $p = 0.3$ (0.3131 to 0.3489). At $p = 0$ the difference was zero (Table II). These intervals describe variation across simulated seeds under this design. They do not show that the same effect sizes would appear in real agent systems.

TABLE II. PAIRED SEED-LEVEL UNCERTAINTY

p	Mean paired diff.	95% t interval	Seeds
0.0	0.0000	[0.0000, 0.0000]	30
0.1	0.1070	[0.0944, 0.1196]	30
0.2	0.2067	[0.1888, 0.2245]	30
0.3	0.3310	[0.3131, 0.3489]	30

D. Any-Error Failure

The any-error failure rate was the same in both conditions at every base rate: 0.0000 at $p = 0$, 0.3360 at $p = 0.1$, 0.5787 at $p = 0.2$ and 0.7447 at $p = 0.3$. At $p = 0.1$, for instance, 1,008 of 3,000 workflows failed in each condition. This equality follows from the metric definition. Both conditions use probability p for the first stage, and a workflow is labeled failed as soon as any stage errs, so raising the probability of later errors cannot change the label of a workflow that has already had its first error.

E. Analytical Consistency Check

The empirical propagation means (0.4917, 0.9953 and 1.5100 for $p = 0.1$, 0.2 and 0.3) sit close to the theoretical expectations (0.5107, 1.0214 and 1.5321). The independent means (0.3847, 0.7887 and 1.1790) are likewise close to $4p$ (0.4000, 0.8000 and 1.2000). The small gaps are consistent with finite Monte Carlo sampling. This supports agreement between the stated mechanism and the observed aggregate behavior, but it is not an independent validation of real-world assumptions.

VI. DISCUSSION

Within the simulator, the results support H1 and H2. Propagation raises the number of stage errors that follow an upstream error, and the gap widens as the base probability rises. This matches the mechanism built into the simulation: once an error occurs, the next stage is exposed to a higher error probability. It also shows why a count of stage errors can reveal degradation that a simple binary outcome hides.

H3 holds by construction. The metric F records whether at least one stage has failed, and because the propagation rule leaves the probability of the first error unchanged, it leaves the probability of at least one error unchanged. That is not evidence that propagation is harmless. It shows that a metric can be blind to further downstream damage once it has crossed a threshold. Future studies should separate first-error incidence, stage-error accumulation, terminal answer correctness, recovery and claim-level evidence support.

The findings fit the prior review's argument that research automation should be evaluated at intermediate stages and not only by final task completion. A practical system could preserve stage outputs, source identifiers and claim-to-evidence links; run schema and provenance checks at handoffs; and use an independent verifier with direct access to primary sources. Retrieval and reflection methods offer useful design ingredients [7]–[9], but this simulation tests no mitigation. The next experiment should add a verification intervention and measure whether it reduces later-stage errors and improves terminal correctness.

VII. THREATS TO VALIDITY AND LIMITATIONS

Construct validity. A simulated Bernoulli error is a thin abstraction of a much richer phenomenon. Real agent errors differ in type, severity, detectability and dependence; a wrong source, a parsing failure and a reasoning error are not interchangeable. The model counts every stage error equally and ignores what an error means.

Internal validity. The increment $\delta = 0.30$ is a design assumption, not a calibrated parameter, and only four stages and four base rates were examined. The model uses a one-step dependence rule, where the next stage depends only on whether the immediately preceding stage erred. It does not model persistent error state, multiple upstream causes, correlated agents, correction or feedback loops.

Measurement validity. The primary metric counts stage errors. The secondary metric F flags any error and is not a measure of final answer correctness. No terminal output is generated, no claims are extracted and no citation-support score is computed, so the study cannot estimate manuscript correctness, citation accuracy, research quality or user impact.

External validity. No LLM, multi-agent framework, live scholarly search API or HotpotQA dataset was run. Results must not be presented as empirical performance measurements of deployed agentic AI systems. The environment log records a GPU, but the simulation does not need one.

Statistical validity. The paired intervals use seed-level differences and so preserve the shared-seed design. They describe uncertainty across 30 simulated seeds, not across a population of

real research workflows. Because the rule is fixed and sampling is pseudorandom, the results are conditional on the specified generator and protocol.

VIII. REPRODUCIBILITY AND PROVENANCE

The project archive contains the Phase III notebooks, the Phase IV evaluation notebook and script, the raw multi-seed CSV, summary CSV files, validation JSON, environment logs and result figures. The raw-data dimensions and cell counts were checked directly. The key configuration is: four stages; $p \in \{0.0, 0.1, 0.2, 0.3\}$; $\delta = 0.30$; 30 seeds from 2026 through 2055; 100 trials per seed and condition; and outcome definitions $S = \Sigma E_i$ and $F = 1(S > 0)$.

To reproduce the results, load the archived multi_seed_results.csv, verify the six required columns, check the 24,000-row total and 3,000 rows per rate-condition cell, validate the outcome label and recompute the group means. The project includes evaluate_phaseiv.py and notebook-based validation logic. A clean-clone reproduction has not been completed.

AI assistance was used to help organize evaluation materials, manuscript structure, documentation and interpretation checks. The code and the archived CSV, not AI-generated prose, are the evidence for the findings, and the author is responsible for inspecting the code, verifying the sources, checking the actual outputs and approving every scientific claim.

IX. CONCLUSION AND FUTURE WORK

This paper presented a controlled simulation of conditional error propagation in a four-stage research-oriented workflow. Across 24,000 records, propagation increased the mean number of stage errors at every non-zero base rate, with paired mean differences rising from 0.107 at $p = 0.1$ to 0.331 at $p = 0.3$. The binary any-error failure rate stayed equal between conditions because the outcome is defined by the first error. The findings show why metrics should capture accumulated downstream damage rather than rely only on a thresholded failure indicator.

The contribution is a transparent, reproducible simulation analysis tied to a prior literature review. It is not a new agent architecture and not a real-world benchmark result. Future work should vary the propagation increment and workflow length, add error recovery and independent verification, define terminal-output correctness, and then validate the model on real agent execution traces or a controlled benchmark. Each stage should keep its provenance, so that a final claim can be traced to its source and the point where an error entered the workflow can be identified.

APPENDIX: EXPERIMENTAL CONFIGURATION

Stages: planning, retrieval, evidence extraction and verification. Base error probabilities: 0.0, 0.1, 0.2 and 0.3. Propagation increment: $\delta = 0.30$, capped at 1.0. Seeds: 2026–2055. Trials: 100 per seed, rate and condition. Total raw records: 24,000. Primary outcome: stage-error count. Secondary outcome: any-error indicator.

REFERENCES

[1] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023, arXiv:2210.03629.

[2] J. Wei *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," in *Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 24824–24837.

[3] X. Wang *et al.*, "Self-consistency improves chain of thought reasoning in language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.

[4] S. Yao *et al.*, "Tree of thoughts: Deliberate problem solving with large language models," in *Adv. Neural Inf. Process. Syst.*, vol. 36, 2023.

[5] T. Schick *et al.*, "Toolformer: Language models can teach themselves to use tools," in *Adv. Neural Inf. Process. Syst.*, vol. 36, 2023.

[6] Y. Qin *et al.*, "ToolLLM: Facilitating large language models to master 16,000+ real-world APIs," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[7] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 9459–9474.

[8] A. Asai *et al.*, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[9] N. Shinn *et al.*, "Reflexion: Language agents with verbal reinforcement learning," in *Adv. Neural Inf. Process. Syst.*, vol. 36, 2023.

[10] Q. Wu *et al.*, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," in *Proc. Conf. Lang. Model. (COLM)*, 2024.

[11] S. Hong *et al.*, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[12] J. S. Park *et al.*, "Generative agents: Interactive simulacra of human behavior," in *Proc. 36th Annu. ACM Symp. User Interface Softw. Technol. (UIST)*, 2023.

[13] G. Wang *et al.*, "Voyager: An open-ended embodied agent with large language models," arXiv:2305.16291, 2023.

[14] X. Liu *et al.*, "AgentBench: Evaluating LLMs as agents," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[15] S. Zhou *et al.*, "WebArena: A realistic web environment for building autonomous agents," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[16] X. Deng *et al.*, "Mind2Web: Towards a generalist agent for the web," in *Adv. Neural Inf. Process. Syst.*, vol. 36, 2023.

[17] G. Mialon *et al.*, "GAIA: A benchmark for general AI assistants," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.

[18] C. E. Jimenez *et al.*, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[19] T. Gao, H. Yen, J. Yu, and D. Chen, "Enabling large language models to generate text with citations," in *Proc. Conf. Empirical Methods Nat. Lang. Process. (EMNLP)*, 2023, pp. 6465–6488.

[20] S. Min *et al.*, "FActScore: Fine-grained atomic evaluation of factual precision in long form text generation," in *Proc. Conf. Empirical Methods Nat. Lang. Process. (EMNLP)*, 2023, pp. 12076–12100.